МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 6

Тема: **Структурные паттерны: Заместитель**

Дисциплина: **«Паттерны проектирования»**

**Выполнила:**

Студентка группы ПИЖ-б-о-23-2
направление подготовки 09.03.04
«Программная инженерия»

Силина Оксана Романовна

**Проверил:**

Ассистент ДЦРСИЭ

Белик А. И.

Ставрополь 2026

**Цель работы:** изучить принцип работы структурного паттерна проектирования «Заместитель» и реализовать прикладную задачу расчёта траектории движения тела с использованием данного паттерна.

**Задача:** разработать приложение для расчёта параметров движения тела, брошенного под углом к горизонту, с использованием структурного паттерна проектирования «Заместитель».

Выбранный паттерн:

- Заместитель — для контроля доступа к объекту расчёта траектории: кэширование результатов, логирование обращений и ленивая инициализация тяжёлого объекта.

**Индивидуальный вариант:** 3. Разработка приложения «Траектория движения»

**Репозиторий:** https://github.com/tsukuri1/patterns

## 1. Описание паттерна «Заместитель»

Заместитель — это структурный паттерн проектирования, который предоставляет объект-заменитель, контролирующий доступ к другому объекту. Заместитель имеет тот же интерфейс, что и реальный объект, поэтому клиентский код не замечает подмены.

Паттерн применяется в нескольких вариантах:

- виртуальный заместитель — откладывает создание тяжёлого объекта до первого обращения;
- кэширующий заместитель — сохраняет результаты и возвращает их при повторных запросах;
- логирующий заместитель — записывает обращения к объекту;
- защищающий заместитель — проверяет права доступа.

Применение в задаче:

Интерфейс TrajectoryService задаёт методы calculate(v0, angle) и points(v0, angle, steps). Реальный объект RealTrajectoryService выполняет расчёт. Заместитель TrajectoryProxy реализует тот же интерфейс, но:

- создаёт реальный объект только при первом обращении (виртуальный заместитель);
- кэширует результаты по ключу (v0, angle) (кэширующий заместитель);
- записывает в журнал каждое обращение — попадание в кэш или реальный расчёт (логирующий заместитель).

Клиентский код (интерфейс приложения) работает только с TrajectoryService и не знает, что за ним стоит заместитель.

## 2. Структура приложения

Приложение состоит из следующих компонентов:

| Компонент | Назначение |
|---|---|
| TrajectoryService | Общий интерфейс сервиса расчёта |
| RealTrajectoryService | Реальный объект — выполняет расчёт |
| TrajectoryProxy | Заместитель — кэширует, логирует, лениво создаёт реальный объект |
| TrajectoryApp | Графический интерфейс приложения |

## 3. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

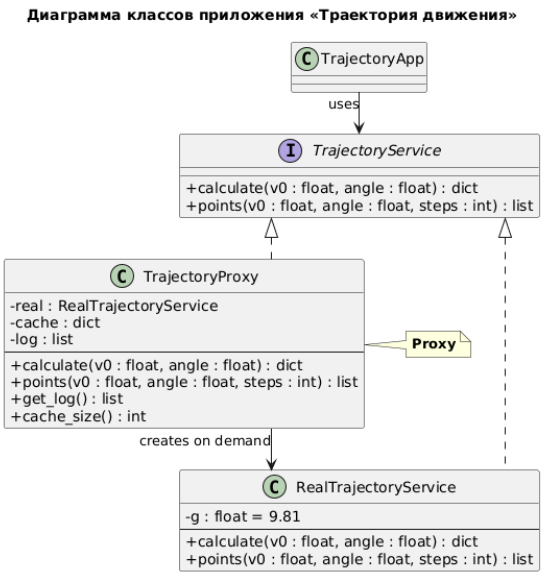



Рисунок 1 — Диаграмма классов приложения «Траектория движения»

Диаграмма содержит интерфейс TrajectoryService, реальный объект RealTrajectoryService, заместитель TrajectoryProxy и класс TrajectoryApp — интерфейс приложения. Заместитель и реальный объект реализуют один интерфейс, поэтому клиент работает с ними одинаково.

## 4. Реализация паттерна «Заместитель»

TrajectoryProxy реализует тот же интерфейс TrajectoryService, что и реальный сервис. Внутри он хранит ссылку на реальный объект, создаваемую при первом обращении, кэш результатов и журнал обращений. Метод calculate() сначала проверяет кэш: если результат для таких параметров уже есть, он возвращается без вычислений; иначе вызывается реальный сервис, а результат сохраняется в кэш. Каждое обращение записывается в журнал с пометкой «cache hit» или «real call».

```
import math


class TrajectoryService:
    """Общий интерфейс сервиса расчёта."""
    def calculate(self, v0, angle):
```

```
        raise NotImplementedError

    def points(self, v0, angle, steps):
        raise NotImplementedError


class RealTrajectoryService(TrajectoryService):
    """Реальный объект — выполняет расчёт."""
    def __init__(self, g=9.81):
        self.g = g

    def calculate(self, v0, angle):
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        a = math.radians(angle)
        T = 2 * v0 * math.sin(a) / self.g
        H = (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)
        L = (v0 ** 2) * math.sin(2 * a) / self.g
        return {"time": T, "height": H, "range": L}

    def points(self, v0, angle, steps):
        res = self.calculate(v0, angle)
        T = res["time"]
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class TrajectoryProxy(TrajectoryService):
    """Заместитель: кэш, лог, ленивое создание реального объекта."""
    def __init__(self):
        self._real = None
        self._cache = {}
        self._log = []

    def _get_real(self):
        if self._real is None:
```

```
            self._real = RealTrajectoryService()
        return self._real

    def calculate(self, v0, angle):
        key = (round(v0, 3), round(angle, 3))
        if key in self._cache:
            self._log.append(f"CACHE HIT  calculate({v0}, {angle})")
            return self._cache[key]
        self._log.append(f"REAL CALL  calculate({v0}, {angle})")
        result = self._get_real().calculate(v0, angle)
        self._cache[key] = result
        return result

    def points(self, v0, angle, steps):
        key = (round(v0, 3), round(angle, 3), steps)
        if key in self._cache:
            self._log.append(f"CACHE HIT  points({v0}, {angle}, {steps})")
            return self._cache[key]
        self._log.append(f"REAL CALL  points({v0}, {angle}, {steps})")
        result = self._get_real().points(v0, angle, steps)
        self._cache[key] = result
        return result

    def get_log(self):
        return list(self._log)

    def cache_size(self):
        return len(self._cache)
```

## 5. Графический интерфейс

Интерфейс позволяет ввести параметры и выполнить расчёт. При повторном нажатии «Рассчитать» с теми же параметрами заместитель вернёт результат из кэша, что видно по журналу: вместо REAL CALL появится CACHE HIT. Кнопка «Показать журнал заместителя» выводит все обращения, а кнопка «Очистить кэш» сбрасывает кэш, чтобы можно было снова увидеть реальный расчёт.

```
import tkinter as tk
from tkinter import messagebox
```

```
class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        root.title("Траектория движения")

        self.proxy = TrajectoryProxy()

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Шагов траектории:").grid(row=2,
column=0, sticky="w")
        self.steps_entry = tk.Entry(root)
        self.steps_entry.insert(0, "50")
        self.steps_entry.grid(row=2, column=1)

        tk.Button(root, text="Рассчитать",
                  command=self.calculate).grid(row=3, column=0,
columnspan=2, pady=5)
        tk.Button(root, text="Показать журнал заместителя",
                  command=self.show_log).grid(row=4, column=0,
columnspan=2)
        tk.Button(root, text="Очистить кэш",
                  command=self.clear_cache).grid(row=5, column=0,
columnspan=2)

        self.result = tk.Text(root, width=55, height=14)
        self.result.grid(row=6, column=0, columnspan=2)

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
        angle = float(self.angle_entry.get())
        steps = int(self.steps_entry.get())
        return v0, angle, steps

    def calculate(self):
```

```
        try:
            v0, angle, steps = self._read_inputs()
            res = self.proxy.calculate(v0, angle)
            pts = self.proxy.points(v0, angle, steps)

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Время полёта: {res['time']:.3f} с\n"
                f"Макс. высота: {res['height']:.3f} м\n"
                f"Дальность: {res['range']:.3f} м\n"
                f"Точек траектории: {len(pts)}\n"
                f"Размер кэша: {self.proxy.cache_size()}\n\n"
                f"Последние записи журнала:\n")
            for line in self.proxy.get_log()[-4:]:
                self.result.insert(tk.END, "  " + line + "\n")
            self.root.update_idletasks()

            import matplotlib.pyplot as plt
            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title("Траектория движения")
            plt.xlabel("X, м")
            plt.ylabel("Y, м")
            plt.grid(True)
            plt.axis('equal')
            plt.show()
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))

    def show_log(self):
        log = self.proxy.get_log()
        text = "\n".join(log) if log else "(журнал пуст)"
        messagebox.showinfo("Журнал заместителя", text)

    def clear_cache(self):
        self.proxy._cache.clear()
        messagebox.showinfo("Кэш", "Кэш заместителя очищен.")
```

## 6. Результаты работы программы

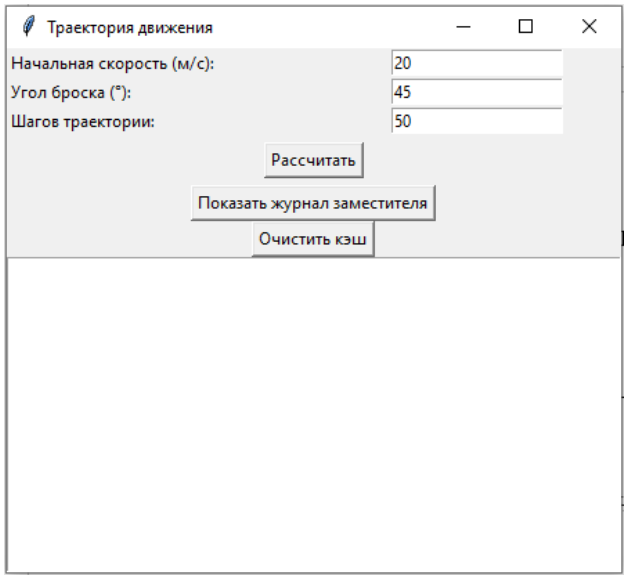



Рисунок 2 — Главное окно приложения

Пользователь вводит начальную скорость, угол броска и число шагов траектории.

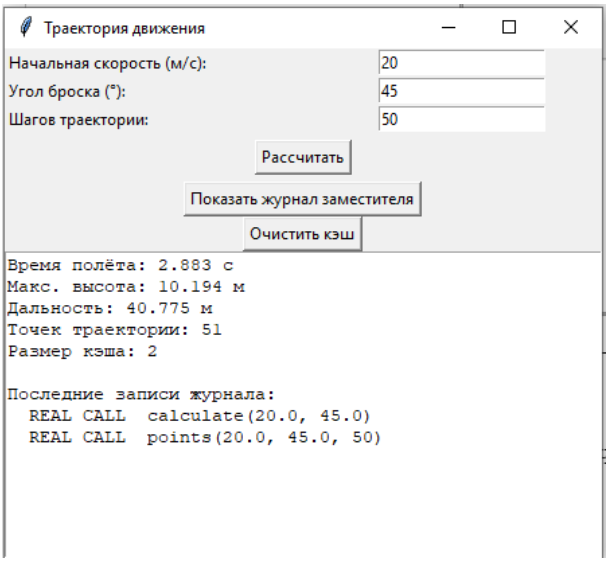



Рисунок 3 — Первый расчёт (реальный вызов)

При v0 = 20 м/с, угол = 45°, шагов = 50:

- Время полёта: 2.883 с
- Макс. высота: 10.194 м

- Дальность: 40.774 м
- Точек траектории: 51
- Размер кэша: 2 (закэшированы и calculate, и points)

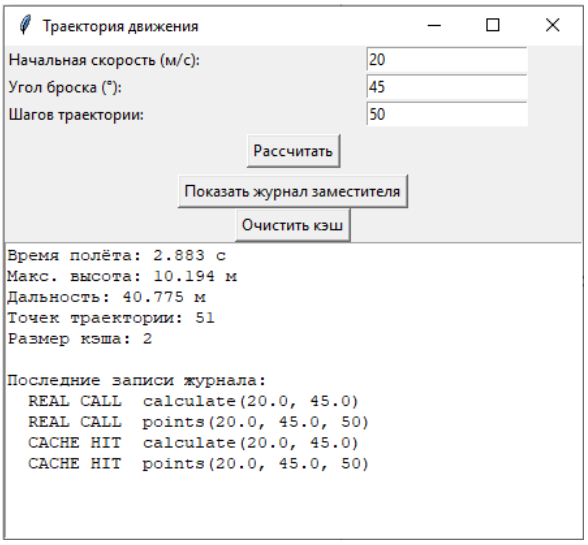



Рисунок 4 — Повторный расчёт с теми же параметрами (попадание в кэш)

Размер кэша не изменился — вычисления не повторялись.

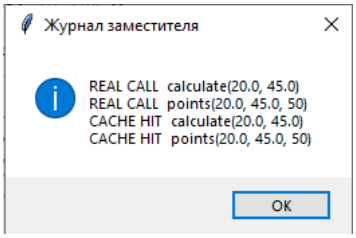



Рисунок 5 — Окно «Показать журнал заместителя»

Показывает все обращения: и реальные вызовы, и попадания в кэш.

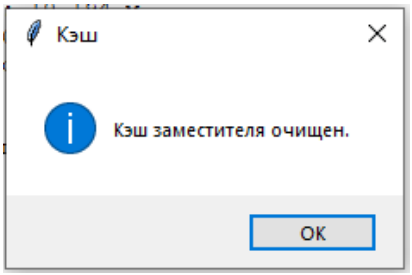



Рисунок 6 — После очистки кэша

Кнопка «Очистить кэш» сбрасывает кэш. Повторный расчёт снова покажет REAL CALL, что подтверждает: реальный объект вызывается только при отсутствии готового результата.

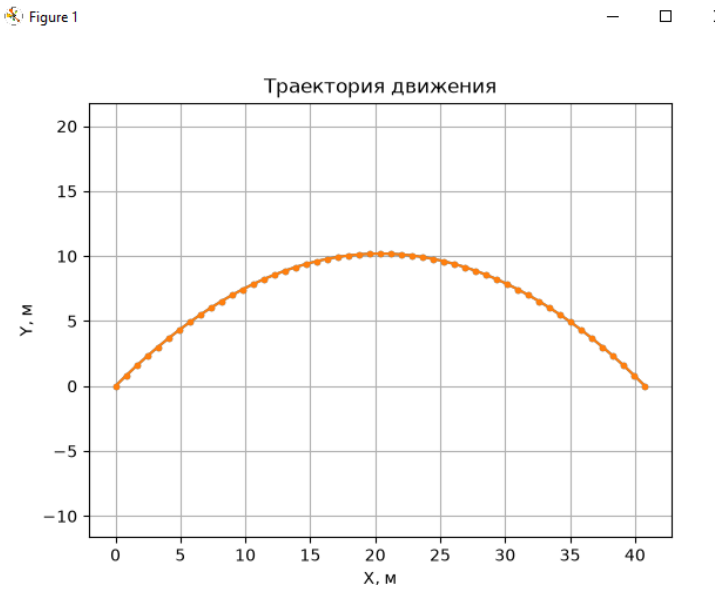



Рисунок 7 — График траектории

Строится по точкам, полученным от заместителя.

## 6. Исходный код

```
import math
import tkinter as tk
from tkinter import messagebox


class TrajectoryService:
    """Общий интерфейс сервиса расчёта."""
    def calculate(self, v0, angle):
        raise NotImplementedError

    def points(self, v0, angle, steps):
        raise NotImplementedError


class RealTrajectoryService(TrajectoryService):
    """Реальный объект — выполняет расчёт."""
```

```
        def __init__(self, g=9.81):
            self.g = g

        def calculate(self, v0, angle):
            if v0 <= 0:
                raise ValueError("Скорость должна быть > 0")
            if not (0 < angle < 90):
                raise ValueError("Угол должен быть в диапазоне (0; 90)")
            a = math.radians(angle)
            T = 2 * v0 * math.sin(a) / self.g
            H = (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)
            L = (v0 ** 2) * math.sin(2 * a) / self.g
            return {"time": T, "height": H, "range": L}

        def points(self, v0, angle, steps):
            res = self.calculate(v0, angle)
            T = res["time"]
            a = math.radians(angle)
            pts = []
            for i in range(steps + 1):
                t = T * i / steps
                x = v0 * math.cos(a) * t
                y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
                pts.append((round(x, 3), round(y, 3)))
            return pts


    class TrajectoryProxy(TrajectoryService):
        """Заместитель: кэш, лог, ленивое создание реального объекта."""
        def __init__(self):
            self._real = None
            self._cache = {}
            self._log = []

        def _get_real(self):
            if self._real is None:
                self._real = RealTrajectoryService()
            return self._real

        def calculate(self, v0, angle):
            key = (round(v0, 3), round(angle, 3))
            if key in self._cache:
                self._log.append(f"CACHE HIT  calculate({v0}, {angle})")
                return self._cache[key]
```

```
            self._log.append(f"REAL CALL  calculate({v0}, {angle})")
            result = self._get_real().calculate(v0, angle)
            self._cache[key] = result
            return result

        def points(self, v0, angle, steps):
            key = (round(v0, 3), round(angle, 3), steps)
            if key in self._cache:
                self._log.append(f"CACHE HIT  points({v0}, {angle}, {steps})")
                return self._cache[key]
            self._log.append(f"REAL CALL  points({v0}, {angle}, {steps})")
            result = self._get_real().points(v0, angle, steps)
            self._cache[key] = result
            return result

        def get_log(self):
            return list(self._log)

        def cache_size(self):
            return len(self._cache)

    class TrajectoryApp:
        def __init__(self, root):
            self.root = root
            root.title("Траектория движения")

            self.proxy = TrajectoryProxy()

            tk.Label(root, text="Начальная скорость (м/с):").grid(row=0, column=0, sticky="w")
            self.v0_entry = tk.Entry(root)
            self.v0_entry.insert(0, "20")
            self.v0_entry.grid(row=0, column=1)

            tk.Label(root, text="Угол броска (°):").grid(row=1, column=0, sticky="w")
            self.angle_entry = tk.Entry(root)
            self.angle_entry.insert(0, "45")
            self.angle_entry.grid(row=1, column=1)

            tk.Label(root, text="Шагов траектории:").grid(row=2, column=0, sticky="w")
            self.steps_entry = tk.Entry(root)
```

```
        self.steps_entry.insert(0, "50")
        self.steps_entry.grid(row=2, column=1)

        tk.Button(root, text="Рассчитать",
                  command=self.calculate).grid(row=3, column=0,
columnspan=2, pady=5)
        tk.Button(root, text="Показать журнал заместителя",
                  command=self.show_log).grid(row=4, column=0,
columnspan=2)
        tk.Button(root, text="Очистить кэш",
                  command=self.clear_cache).grid(row=5, column=0,
columnspan=2)

        self.result = tk.Text(root, width=55, height=14)
        self.result.grid(row=6, column=0, columnspan=2)

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
        angle = float(self.angle_entry.get())
        steps = int(self.steps_entry.get())
        return v0, angle, steps

    def calculate(self):
        try:
            v0, angle, steps = self._read_inputs()
            res = self.proxy.calculate(v0, angle)
            pts = self.proxy.points(v0, angle, steps)

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Время полёта: {res['time']:.3f} с\n"
                f"Макс. высота: {res['height']:.3f} м\n"
                f"Дальность: {res['range']:.3f} м\n"
                f"Точек траектории: {len(pts)}\n"
                f"Размер кэша: {self.proxy.cache_size()}\n\n"
                f"Последние записи журнала:\n")
            for line in self.proxy.get_log()[-4:]:
                self.result.insert(tk.END, "  " + line + "\n")
            self.root.update_idletasks()

            import matplotlib.pyplot as plt
            xs = [p[0] for p in pts]
            ys = [p[1] for p in pts]
            plt.plot(xs, ys, marker='o', markersize=3)
            plt.title("Траектория движения")
```

```
                plt.xlabel("X, м")
                plt.ylabel("Y, м")
                plt.grid(True)
                plt.axis('equal')
                plt.show()
            except ValueError as e:
                messagebox.showerror("Ошибка ввода", str(e))
            except Exception as e:
                messagebox.showerror("Ошибка", str(e))

        def show_log(self):
            log = self.proxy.get_log()
            text = "\n".join(log) if log else "(журнал пуст)"
            messagebox.showinfo("Журнал заместителя", text)

        def clear_cache(self):
            self.proxy._cache.clear()
            messagebox.showinfo("Кэш", "Кэш заместителя очищен.")


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
root.mainloop()
```

**Вывод:**

В ходе выполнения лабораторной работы было разработано приложение «Траектория движения», реализующее расчёт параметров движения тела, брошенного под углом к горизонту.

В приложении применён структурный паттерн проектирования «Заместитель». Класс TrajectoryProxy реализует тот же интерфейс TrajectoryService, что и реальный сервис RealTrajectoryService, и выполняет три функции:

- ленивая инициализация — реальный объект создаётся только при первом обращении;
- кэширование — результаты расчёта сохраняются по ключу (v0, angle) и возвращаются при повторных запросах без повторных вычислений;

- логирование — каждое обращение фиксируется в журнале с пометкой «REAL CALL» или «CACHE HIT».

Клиентский код работает с интерфейсом TrajectoryService и не знает, что за ним стоит заместитель. Это делает подмену прозрачной и позволяет добавлять новые варианты заместителей (например, защищающий или синхронизирующий) без изменения приложения.

Таким образом, была изучена практическая реализация паттерна «Заместитель» и получено работоспособное приложение с графическим интерфейсом для расчёта и визуализации траектории движения тела.